Jone Gorman AI 学习指南：从零开始，动手入门

2026-10-19

目录

1. 认识计算机 **1**

2. 认识操作系统 **2**

3. 学习 Python **2**

4. 管理开发环境 **2**

5. 从机器学习走向深度学习 **3**

6. 理解大语言模型 **3**

7. 强化学习：从基础到 PPO **4**

8. 学习 Agent 与应用 **4**

9. 把学习变成可持续的习惯 **5**

昂贵的课程、密集的新名词，以及不断出现的技术热点，很容易让刚开始接触 AI 的人感到焦虑。这份指南从一个预算有限、计算机基础薄弱的学习者出发，把“我要学 AI”拆成可以逐步完成的事情。

从认识电脑，到写出第一段 Python，再到复现模型和完成一个小项目，每一步都可以留下自己的代码与记录。课程是入口，动手实践才让这些知识真正连接起来。

1. 认识计算机

先理解计算机的基本组成，区分 CPU、内存、存储和 GPU，知道程序、文件以及硬件之间大致如何配合。这一阶段不需要先购买高性能设备，也不需要一次掌握所有计算机知识。

- 计算机入门视频

动手记录：查看自己设备的处理器、内存和磁盘容量，整理常用文件的位置。内存和磁盘存储是不同

概念；安装软件占用的主要是存储空间。

2. 认识操作系统

从熟悉的 Windows 开始，逐渐了解操作系统、终端、目录和文件权限。在需要 Linux 的课程或项目里，再学习相应操作。双系统是原文提供的一条实践路线，也可以先在虚拟机或 WSL 中练习。

- 双系统安装教程
- Linux 基础操作

动手记录：在终端中完成查看当前目录、切换目录、创建文件夹和运行程序，并理解自己输入的命令做了什么。

3. 学习 Python

用一门语言建立编程基础，比同时追逐多个工具更容易开始。这里选择 Python，先学会变量、条件、循环、函数、常用数据结构、模块导入和基本报错排查，再进入机器学习代码。

- Python 与 VS Code 环境搭建
- Python 基础课程

动手记录：独立写一个读取文件、处理数据并输出结果的小脚本。跟着视频敲完之后，修改输入或增加一个功能，确认自己能够解释程序的行为。

4. 管理开发环境

项目之间往往需要不同的 Python 和依赖版本。Miniconda 提供较小的初始安装，可以按项目创建环境；随后使用 conda 或环境中的 pip 安装所需依赖。它与 Anaconda 的区别不需要用固定的体积倍数来判断，实际占用取决于安装内容。

- Miniconda 入门教程
- conda 官方环境管理文档

原文以 Python 3.10 为例创建第一个环境。这个版本用于展示操作方式，实际项目应匹配课程或依赖文件中的版本要求。

```
conda create -n aienv python=3.10
conda activate aienv
python --version
python -m pip --version
```

动手记录：确认终端、编辑器和安装依赖所使用的是同一个环境，记录项目的 Python 版本及依赖配置。

5. 从机器学习走向深度学习

进入 AI 之后，数学、代码和实验会一起出现。遇到暂时不理解的推导，可以先把一个例子运行起来，再回头连接公式、输入输出与代码实现。不同课程的节奏不同，可以交叉阅读，但不必强求短时间内看完所有内容。

课程与实践材料

- 李沐：动手学深度学习
- Abhishek Thakur：Approaching (Almost) Any Machine Learning Problem，作者仓库
- AAAMLP 中文翻译
- RethinkFun 深度学习教程

Git、GitHub 与设备

写代码的同时学习版本管理：Git 用来记录修改，GitHub 可以托管和分享仓库。先用现有设备运行适合其规模的例子，再根据项目需求判断是否需要 GPU。原文中的装机视频作为硬件知识补充保留。

- GitHub 介绍
- Git 介绍
- 硬件茶谈装机教程

动手记录：完成一个有数据划分、评价指标和结果记录的模型实验，并将自己的实现提交到 Git 仓库。

6. 理解大语言模型

从 Transformer 开始，把注意力机制、张量形状和前向传播联系起来。逐行实现一个教学模型，再阅读 Llama 等架构，能够让论文中的模块不再只是名词。这里的学习顺序是路线安排，不是对技术新旧或性能的排名。

Transformer 与 Llama

- Attention Is All You Need 讲解
- Transformer 的 PyTorch 实现
- Llama 架构解析
- Llama 的 PyTorch 实现

模型结构

- RoPE：旋转位置编码
- MoE：混合专家
- RMSNorm

微调、推理与效率

LoRA 是参数高效微调方法，可以用于监督微调。KV Cache、FlashAttention 和 PagedAttention 则从不同角度处理注意力计算、推理或内存管理，学习时要区分它们解决的问题。

- LoRA 微调
- KV Cache
- FlashAttention
- PagedAttention

动手记录：在教学规模上实现并解释注意力模块，记录关键张量的形状；阅读后续技术时，写清它改变了哪个环节、引入了哪些条件。

7. 强化学习：从基础到 PPO

先理解状态、动作、奖励、策略和价值函数，再学习算法与代码。PPO 是这条路线重点展开的一个方法；算法的实际效果取决于任务、实现和评价设置，不能把它概括为所有问题上的“绝对最优”。

- 王树森：深度强化学习
- PPO 教程
- The 37 Implementation Details of Proximal Policy Optimization

复现 PPO 时，把实现细节与实验记录一起学习：观察训练曲线，记录随机种子、超参数和运行环境。原文提到的 TensorBoard、Weights & Biases（W&B）可以用于实验可视化；先建立能保存、能回看的记录，再追求图表的丰富程度。

原文还收录了一个 W&B 网络连接经验帖，作为社区补充材料；这里保留帖子入口，移除了个人分享参数。帖子内容与可访问性以平台实际页面为准。

区分推理框架与强化学习框架

- vLLM 官方文档：主要面向大模型推理和服务，也可以作为强化学习系统中的生成组件。
- verl 项目仓库 与 官方文档：面向大语言模型强化学习。

动手记录：先完成一个小规模 PPO 实验，保留配置和学习曲线，再去理解 LLM 强化学习系统中的训练、生成与评价分别由哪些组件承担。

8. 学习 Agent 与应用

当你已经能阅读 Python、管理依赖并理解模型基本调用方式，就可以学习 Agent 如何连接工具和外部环境。这是原文路线中的应用入口，可以用一个范围明确的小项目把前面的知识串联起来。

- Hugging Face AI Agents Course：课程入口
- 原文收录的课程直播页面

动手记录：选择一个可以核对结果的任务，实现一次工具调用，记录输入、工具返回和最终输出，再检查模型是否确实使用了返回的信息。

9. 把学习变成可持续的习惯

英语阅读和听力能够帮助你直接接触课程、论文和官方文档。遇到翻译不清楚的内容时，回到原文、代码和实际运行结果，往往比继续收集更多二手解释有效。

原文提到的学习者与创作者包括：

- **Umar Jamil**：算法实现与论文讲解。
- **Andrej Karpathy**：神经网络和语言模型的代码实践。
- **Mu Li（李沐）**：深度学习课程、论文阅读与工程实践。

围绕自己感兴趣的问题检索资料、阅读文档、复现代码，也向他人清楚地描述问题并寻求帮助。保留环境、修改和结果记录，让学习过程可以回看和继续。

LLM 是这条路线的重要部分，AI 的范围也更广。随着技术热点变化，信息检索、理解问题和动手实现的能力，仍然能够支撑下一次学习。

勇敢开始，与志同道合的人交流，保持专注，也保持好奇。**Get your hands dirty.**